



Universidad
Tecnológico

Actividad 2

Centro de operaciones de un sistema



Nombre: Leonel Eduardo Pérez Sáenz

Nombre del Profesor (a): Edgar Ivan Patricio Aizpuru

Materia: Estructura de datos

Matricula: AL07128560

1. Introducción

Se desarrolló un programa en Java que utiliza una **Pila (Stack)** y una **Cola (Queue)** implementadas mediante nodos y referencias.

La Pila administra el historial de acciones utilizando el principio **LIFO**, mientras que la Cola administra tareas pendientes utilizando el principio **FIFO**.

2. Estructuras utilizadas

Node

La clase Node representa cada elemento de las estructuras y contiene:

- data: almacena la acción o tarea.
- next: referencia al siguiente nodo.

Stack

La Pila utiliza la referencia top, que apunta al elemento superior.

Sus principales operaciones son:

- push(): agrega una acción.
- pop(): elimina la última acción.
- peek(): consulta la última acción sin eliminarla.
- isEmpty(): verifica si está vacía.
- size(): muestra la cantidad de elementos.

La Pila trabaja con **LIFO (Last In, First Out)**.

Queue

La Cola utiliza las referencias front y rear.

- enqueue(): agrega una tarea al final.
- dequeue(): procesa y elimina la primera tarea.
- peek(): consulta la siguiente tarea.
- isEmpty(): verifica si está vacía.
- size(): muestra la cantidad de tareas.

La Cola trabaja con **FIFO (First In, First Out)**.

3. Funcionamiento

Stack

Ejemplo de elementos agregados:

A

B

C

D

Al retirarlos:

D

C

B

A

Esto demuestra el comportamiento **LIFO**.

```
=====
                        CENTRO DE OPERACIONES
=====
1. Registrar acción
2. Deshacer última acción
3. Ver última acción
4. Mostrar historial

5. Agregar tarea
6. Procesar siguiente tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes

9. Mostrar estado del sistema
0. Salir
=====
Seleccione una opción: 4

===== HISTORIAL DE ACCIONES =====
TOP
|
|-- Accion C
|-- Accion B
|-- Accion A
|
FIN
```

Queue

Ejemplo de elementos agregados:

A

B

C

D

Al procesarlos:

A

B

C

D

Esto demuestra el comportamiento **FIFO**.

```
1. Registrar acción
2. Deshacer última acción
3. Ver última acción
4. Mostrar historial

5. Agregar tarea
6. Procesar siguiente tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes

9. Mostrar estado del sistema
0. Salir
```

```
=====
Seleccione una opción: 8
```

```
===== TAREAS PENDIENTES =====
```

```
FRONT
```

```
|
```

```
|-- Tarea B
```

```
|-- Tarea C
```

```
|-- Tarea D
```

```
|
```

```
REAR
```

4. Pruebas realizadas

Se realizaron las siguientes pruebas:

Stack

- Agregar acciones mediante push().
- Consultar la última acción mediante peek().
- Deshacer una acción mediante pop().
- Comprobar size().
- Comprobar isEmpty().
- Probar la Stack vacía.

```
=====
                        CENTRO DE OPERACIONES
=====
1. Registrar acción
2. Deshacer última acción
3. Ver última acción
4. Mostrar historial

5. Agregar tarea
6. Procesar siguiente tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes

9. Mostrar estado del sistema
0. Salir
=====
Seleccione una opción: 4
El historial está vacío.
```

Queue

- Agregar tareas mediante enqueue().
- Consultar la siguiente tarea mediante peek().
- Procesar tareas mediante dequeue().
- Comprobar size().

- Comprobar isEmpty().
- Probar la Queue vacía.

```
=====
                          CENTRO DE OPERACIONES
=====
1. Registrar acción
2. Deshacer última acción
3. Ver última acción
4. Mostrar historial

5. Agregar tarea
6. Procesar siguiente tarea
7. Ver siguiente tarea
8. Mostrar tareas pendientes

9. Mostrar estado del sistema
0. Salir
=====
Seleccione una opción: 6

No hay tareas pendientes.
```

5. Resultados

Las pruebas realizadas demostraron que:

Prueba	Resultado
Stack — LIFO	Correcto
Queue — FIFO	Correcto
push() / pop()	Correcto
peek()	Correcto
enqueue() / dequeue()	Correcto
size()	Correcto
isEmpty()	Correcto

Stack vacía	Controlada
-------------	------------

Queue vacía	Controlada
-------------	------------

6. Conclusión

La implementación permitió comprobar la diferencia entre una Pila y una Cola.

La **Stack** utiliza LIFO, por lo que el último elemento agregado es el primero en salir. La **Queue** utiliza FIFO, por lo que el primer elemento agregado es el primero en salir.

También se comprobó que ambas estructuras manejan correctamente los casos en los que se encuentran vacías.